

Kubernetes Operations

SRE & Troubleshooting Masterclass

*Cluster Internals · Ports · Filesystem · Logging · Failure Scenarios · Node Debugging ·
Production Operations*

9 Phases — Beginner to Senior SRE

Phase 1: Live Cluster · Phase 2: Ports & Networking · Phase 3: Filesystem Layout · Phase 4:
Logging & Mindset

Phase 5: YAML & Cluster Reaction · Phase 6: Blunder Investigation · Phase 7: Resources ·
Phase 8: Node Internals · Phase 9: Production Ops

Phase 1 — Understanding a Live Kubernetes Cluster

You have just SSH'd into a Kubernetes cluster for the first time as the new SRE. The cluster is live, applications are running, and you need to understand what you are looking at. This phase is your orientation — understanding what every component is, where it lives, and why it exists.

1.1 The Two-Tier Architecture — Control Plane and Data Plane

Kubernetes has a fundamental architectural split: the control plane (brain) and the data plane (muscles). Understanding this split is the foundation of all troubleshooting.



1.2 First Commands as a New SRE — Cluster Orientation

```
# What nodes are in this cluster?
kubectl get nodes -o wide

# What namespaces exist?
kubectl get namespaces
```

```

# What is running in kube-system?
kubectl get pods -n kube-system -o wide

# What version is this cluster?
kubectl version

# Cluster info
kubectl cluster-info

# Component status (deprecated but still useful)
kubectl get componentstatuses

```

1.3 The Critical Namespaces

Namespace	What Lives Here and Why
kube-system	The most critical namespace. All Kubernetes system components run here: kube-apiserver, etcd, kube-controller-manager, kube-scheduler (as static pods on control plane nodes), CoreDNS (as Deployment), kube-proxy (as DaemonSet), CNI plugin pods (as DaemonSet), metrics-server, CSI drivers. Never delete resources from kube-system without understanding the impact.
default	The fallback namespace for resources created without an explicit namespace. In production clusters, this should be empty or contain only cluster-level test resources. Application teams should use dedicated namespaces.
kube-public	Contains one ConfigMap: cluster-info. This namespace is readable by all users including unauthenticated ones. It exists for cluster discovery — the kubeconfig bootstrap token process uses it.
kube-node-lease	Contains Lease objects — one per node. Kubelet updates its Lease object every 10 seconds (default) as a heartbeat. The controller-manager uses these Leases to determine node health. If a Lease is not updated within the node-monitor-grace-period (default 40s), the node is marked NotReady.
monitoring	(Common convention) Prometheus, Grafana, AlertManager. Not a default namespace — created by teams.
ingress-nginx	(Common convention) NGINX Ingress Controller DaemonSet/Deployment.
cert-manager	cert-manager operator and its CRDs.
argocd	ArgoCD GitOps controller and UI.

1.4 Static Pods vs Regular Pods — Critical Distinction

This is one of the most important concepts for cluster troubleshooting. Control plane components on master nodes run as static pods — they behave very differently from regular pods.

Pod Type	How it Works and Implications
Regular Pod	Created by Kubernetes controllers (Deployment, StatefulSet, DaemonSet). Stored in etcd. If deleted, the controller recreates it. Managed by kube-apiserver.
Static Pod	Defined by a YAML file in /etc/kubernetes/manifests/ on the node. Kubelet reads this directory and creates the pod directly — without talking to the API server. If the API server is down, static pods still run. If you edit the manifest file, kubelet detects the change and recreates the pod automatically. If you delete the pod via kubectl, kubelet immediately recreates it (the manifest file still exists).
DaemonSet Pod	Managed by DaemonSet controller. Runs exactly one pod per node. Used for node-level services: kube-proxy, CNI plugin, node-exporter, log collectors, CSI node driver.

Static Pod Manifests Location

```
# On a kubeadm cluster, control plane static pods are here:
ls /etc/kubernetes/manifests/
# Output:
# etcd.yaml
# kube-apiserver.yaml
# kube-controller-manager.yaml
# kube-scheduler.yaml
```

```
# Kubelet watches this directory. Edit a file → pod restarts automatically
# This is how you configure control plane components
```

1.5 Control Plane Component Deep Dive

Component	Role and What Breaks Without It
kube-apiserver	The front door to everything. Every kubectl command, every controller, every kubelet communicates through the API server. If it dies: no kubectl commands work, no new pods can be scheduled, controllers cannot reconcile, but EXISTING pods keep running (kubelet runs them independently). This is the most critical component.
etcd	The database of Kubernetes. Every object (pod, service, configmap, secret) is stored in etcd. If etcd dies: API server

Component	Role and What Breaks Without It
	cannot serve reads or writes, cluster appears dead but existing pods keep running. If etcd data is corrupted or lost without backup: cluster state is gone, all objects must be recreated.
kube-controller-manager	Runs all the control loops: Deployment controller, ReplicaSet controller, Node controller, Job controller, etc. If it dies: deployments do not scale, failed pods are not replaced, dead nodes are not evicted. Existing pods keep running, but self-healing stops.
kube-scheduler	Assigns pods to nodes. If it dies: new pods remain in Pending state (Unschedulable). Existing pods keep running. New pods with explicit nodeName set will still start (scheduler is bypassed).
kubelet	Node agent. Runs on every worker node. Responsible for: running pods, health checking pods, reporting node status, mounting volumes. If it dies: node goes NotReady, pods on that node eventually get evicted and rescheduled elsewhere.
kube-proxy	Manages iptables/IPVS rules for Kubernetes Services. If it dies: existing service connections may still work (rules already in iptables), but new services or rule changes will not take effect.
CoreDNS	Internal DNS server. Resolves service names to ClusterIPs. If it dies: service discovery via DNS fails. Pod-to-pod by IP still works. Applications that use service names (most of them) break.

1.6 The kubectl Request Flow — What Actually Happens

```
kubectl get pods -n production
```

1. kubectl reads ~/.kube/config
 - finds cluster endpoint (e.g., https://api.prod.company.com:6443)
 - finds credentials (token or client certificate)
2. kubectl constructs HTTP request:


```
GET https://api.prod.company.com:6443/api/v1/namespaces/production/pods
Authorization: Bearer eyJhbGciOi...
```
3. kube-apiserver (running on control plane node):
 - Authentication: validate token/cert
 - Authorization: RBAC check
 - Send request to etcd:


```
GET /registry/pods/production (etcd key)
```
4. etcd returns pod objects as serialized protobuf

```
5. kube-apiserver deserializes, converts to JSON
   Returns HTTP 200 with pod list

6. kubectl formats and prints the table output
```

1.7 Systemd vs Container — How Components Are Managed

Component	How It Runs and How to Manage It
kubelet	Systemd service on every node. Not a pod. Managed with: <code>systemctl status kubelet</code> , <code>systemctl restart kubelet</code> , <code>journalctl -u kubelet -f</code>
containerd / cri-o	Systemd service. The container runtime. Managed with: <code>systemctl status containerd</code> , <code>journalctl -u containerd</code>
kube-apiserver	Static pod (on kubeadm clusters). Manage by editing <code>/etc/kubernetes/manifests/kube-apiserver.yaml</code>
etcd	Static pod (kubeadm) or systemd service (some setups). On kubeadm: <code>/etc/kubernetes/manifests/etcd.yaml</code>
kube-controller-manager	Static pod on kubeadm. Edit <code>/etc/kubernetes/manifests/kube-controller-manager.yaml</code>
kube-scheduler	Static pod on kubeadm. Edit <code>/etc/kubernetes/manifests/kube-scheduler.yaml</code>
CoreDNS	Regular Deployment in kube-system. Managed with kubectl.
kube-proxy	DaemonSet in kube-system. Managed with kubectl.

Phase 2 — Ports and Networking Deep Dive

Every experienced Kubernetes operator has the critical ports memorized. Port knowledge lets you: verify components are listening, debug firewall issues, trace network paths, and understand security boundaries.

2.1 Complete Kubernetes Port Reference

Component	Port(s)	Description and Who Connects
kube-apiserver	6443	HTTPS API endpoint. All kubectl commands, all controllers, all kubelets connect here. Must be accessible from all nodes and admin machines.
etcd	2379	Client port. kube-apiserver connects here to read/write cluster state. Should ONLY be accessible from control plane nodes.
etcd	2380	Peer port. etcd cluster members communicate with each other for leader election and data replication.
kubelet	10250	HTTPS API. kube-apiserver connects here for: kubectl exec, kubectl logs, kubectl port-forward. Also used by metrics-server.
kubelet	10255	Read-only HTTP (deprecated, disabled by default). Was used for /stats and /metrics without authentication.
kube-scheduler	10259	HTTPS metrics and health. Only accessible on localhost (control plane node).
controller-manager	10257	HTTPS metrics and health. Only accessible on localhost.
kube-proxy	10249 / 10256	Health and metrics endpoints.
CoreDNS	53 UDP/TCP	DNS queries from pods. Pods send DNS queries to the CoreDNS ClusterIP (e.g., 10.96.0.10).
CoreDNS	9153	Prometheus metrics endpoint.
metrics-server	443	HTTPS API. kubectl top uses this via aggregated API.
NGINX Ingress	80 / 443	HTTP/HTTPS ingress. Exposed via NodePort or LoadBalancer.
NodePort services	30000-32767	Range for NodePort services. Accessible on every node's IP.

2.2 Verifying Ports on Nodes

```

# Check what is listening on the control plane node
ss -tlnp
# or
netstat -tlnp

# Expected output on control plane node:
# LISTEN 0 128 0.0.0.0:6443 *:* users:(("kube-apiserver",pid=1234))
# LISTEN 0 128 127.0.0.1:2379 *:* users:(("etcd",pid=5678))
# LISTEN 0 128 0.0.0.0:2380 *:* users:(("etcd",pid=5678))
# LISTEN 0 128 0.0.0.0:10250 *:* users:(("kubelet",pid=9012))

# Check specific port
ss -tlnp | grep 6443

# Check what process owns a port
lsof -i :10250

# Test connectivity to API server from a node
curl -k https://localhost:6443/healthz
# Expected: ok

# Test etcd health
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/healthcheck-client.crt \
  --key=/etc/kubernetes/pki/etcd/healthcheck-client.key \
  endpoint health

```

2.3 Kubernetes Service Networking — How Services Actually Work

When you create a Service, Kubernetes does not create any 'virtual machine' or 'load balancer process'. Instead, it programs iptables (or IPVS) rules on EVERY node that intercept packets destined for the Service ClusterIP and redirect them to one of the backend pod IPs.

```

# How a Service works - iptables path

Client pod (10.244.1.5) sends packet to Service ClusterIP (10.96.100.50:80)

Packet hits the node's iptables PREROUTING chain

kube-proxy has programmed these rules:
-A KUBE-SERVICES -d 10.96.100.50/32 -p tcp --dport 80 \
  -j KUBE-SVC-XXXXXXXXXXXX

-A KUBE-SVC-XXXXXXXXXXXX -m statistic --mode random --probability 0.33 \
  -j KUBE-SEP-POD1 # → Pod 1 (10.244.1.10:8080)
-A KUBE-SVC-XXXXXXXXXXXX -m statistic --mode random --probability 0.50 \
  -j KUBE-SEP-POD2 # → Pod 2 (10.244.2.20:8080)
-A KUBE-SVC-XXXXXXXXXXXX \
  -j KUBE-SEP-POD3 # → Pod 3 (10.244.3.30:8080)

```



```
DNAT: packet destination rewritten from 10.96.100.50:80 to pod IP:port
Packet delivered to selected pod
```

2.4 DNS Debugging — Service Discovery Failures

CoreDNS is the internal DNS server. Every pod is configured with its nameserver pointing to CoreDNS's ClusterIP (typically 10.96.0.10). When a pod makes a DNS query for 'my-service.my-namespace.svc.cluster.local', the query goes to CoreDNS.

```
# Step 1: Can the pod reach CoreDNS?
kubectl exec -it my-pod -- nslookup kubernetes.default
# Should return the kubernetes service ClusterIP

# Step 2: What is the pod's DNS config?
kubectl exec -it my-pod -- cat /etc/resolv.conf
# Expected:
# nameserver 10.96.0.10
# search default.svc.cluster.local svc.cluster.local cluster.local
# options ndots:5

# Step 3: Is CoreDNS running?
kubectl get pods -n kube-system -l k8s-app=kube-dns

# Step 4: CoreDNS logs
kubectl logs -n kube-system -l k8s-app=kube-dns --tail=50

# Step 5: Test DNS from a debug pod
kubectl run dns-debug --image=busybox:1.35 --rm -it -- nslookup my-service.my-namespace

# Step 6: Check CoreDNS ConfigMap
kubectl get configmap coredns -n kube-system -o yaml
```

2.5 Service Unreachable Debugging Flowchart

```
Service is unreachable → start here

1. Can pods reach the Service ClusterIP?
  kubectl exec -it <pod> -- curl http://<clusterIP>:<port>
  NO → Check kube-proxy is running on the node
      → Check iptables rules: iptables -t nat -L KUBE-SERVICES
  YES → DNS issue (name resolution failing, not connectivity)

2. Can pods reach the backend pods directly?
  kubectl get endpoints <service-name>
  # Are there any endpoints listed?
  EMPTY ENDPOINTS → Label selector mismatch between Service and Pods
```

```
→ Check: kubectl describe service <name>
→ Compare selector with pod labels
HAS ENDPOINTS → But service is failing? kube-proxy or iptables issue
```

```
3. Can you curl a backend pod directly?
kubectl exec -it <pod> -- curl http://<pod-ip>:<container-port>
NO → Application not listening on that port
    → Network policy blocking
YES → Service configuration issue (wrong port mapping)
```

2.6 iptables / IPVS — Inspecting Service Rules

```
# List all Kubernetes service NAT rules
iptables -t nat -L KUBE-SERVICES --line-numbers

# Find rules for a specific service ClusterIP
iptables -t nat -L -n | grep 10.96.100.50

# Count total iptables rules (large clusters can have thousands)
iptables -t nat -L | wc -l

# If using IPVS mode
ipvsadm -Ln
ipvsadm -Ln | grep 10.96.100.50
```

Phase 3 — Kubernetes Filesystem and Configuration Layout

An experienced SRE knows the filesystem like the back of their hand. Knowing where every configuration file, certificate, log, and manifest lives is what separates someone who can debug from someone who cannot.

3.1 The Complete Kubernetes Filesystem Map

```
/etc/kubernetes/      ← Main config directory (control plane)
├─ admin.conf          ← kubeconfig for cluster-admin access
├─ scheduler.conf      ← kubeconfig used by kube-scheduler
├─ controller-manager.conf ← kubeconfig used by controller-manager
├─ kubelet.conf        ← kubeconfig used by kubelet
├─ manifests/          ← Static pod manifests (kubelet watches this)
│   ├── etcd.yaml
│   ├── kube-apiserver.yaml
│   ├── kube-controller-manager.yaml
│   └── kube-scheduler.yaml
└─ pki/                ← All TLS certificates
    ├── ca.crt          ← Kubernetes CA (root of trust)
    ├── ca.key           ← CA private key (PROTECT THIS)
    ├── apiserver.crt    ← API server TLS cert
    ├── apiserver.key    ← API server TLS key
    ├── apiserver-kubelet-client.crt ← API-kubelet auth cert
    ├── apiserver-etcd-client.crt ← API-etcd auth cert
    ├── front-proxy-ca.crt ← Front proxy CA
    └─ etcd/
        ├── ca.crt      ← etcd cluster CA
        ├── server.crt   ← etcd server cert
        ├── peer.crt     ← etcd peer cert
        └─ healthcheck-client.crt ← Health check cert
```

```
/var/lib/kubelet/      ← Kubelet working directory
├─ config.yaml         ← Kubelet configuration
├─ kubeconfig          ← Kubelet's identity credentials
├─ pki/                ← Kubelet client certs
│   ├── kubelet.crt
│   └─ kubelet.key
└─ pods/               ← Pod runtime state
    ├── <pod-uid>/
    │   ├── containers/ ← Container state
    │   ├── volumes/    ← Mounted volumes
    │   └─ etc-hosts    ← Pod /etc/hosts
```

```
/etc/cni/net.d/        ← CNI plugin configurations
├─ 10-flannel.conflist ← Flannel CNI config
└─ calico.conflist     ← Calico CNI config
```

```
/opt/cni/bin/          ← CNI plugin binaries
└─ bridge, flannel, calico, ...
```

```
/var/lib/etcd/          ← etcd data directory
└─ member/              ← etcd member data
    └─ snap/             ← Snapshots
        └─ wal/           ← Write-ahead log
```

```
/var/log/containers/    ← Container log symlinks
└─ kube-apiserver-<node> kube-system <id>.log → symlink
```

```
/var/log/pods/          ← Actual container logs
└─ kube-system_kube-apiserver-<node>_<uid>/
    └─ kube-apiserver/
        └─ 0.log          ← Current log
        └─ 1.log          ← Rotated log
```

3.2 Static Pod Manifest — What It Actually Contains

Let us look at a real kube-apiserver static pod manifest to understand what it configures:

```
# /etc/kubernetes/manifests/kube-apiserver.yaml (simplified)
apiVersion: v1
kind: Pod
metadata:
  name: kube-apiserver
  namespace: kube-system
spec:
  hostNetwork: true          # Uses node's network directly
  priorityClassName: system-node-critical
  containers:
  - name: kube-apiserver
    image: registry.k8s.io/kube-apiserver:v1.29.0
    command:
    - kube-apiserver
    - --advertise-address=10.0.0.10
    - --allow-privileged=true
    - --authorization-mode=Node,RBAC
    - --client-ca-file=/etc/kubernetes/pki/ca.crt
    - --etcd-servers=https://127.0.0.1:2379
    - --etcd-cafile=/etc/kubernetes/pki/etcd/ca.crt
    - --tls-cert-file=/etc/kubernetes/pki/apiserver.crt
    - --tls-private-key-file=/etc/kubernetes/pki/apiserver.key
    - --service-cluster-ip-range=10.96.0.0/12
    - --audit-log-path=/var/log/kubernetes/audit.log
    volumeMounts:
    - mountPath: /etc/kubernetes/pki
      name: k8s-certs
      readOnly: true
  volumes:
  - name: k8s-certs
```

```
hostPath:
  path: /etc/kubernetes/pki
```

3.3 What Happens When You Edit a Static Pod Manifest

This is critical operational knowledge. Editing a static pod manifest is the primary way to configure control plane components.

1. You edit /etc/kubernetes/manifests/kube-apiserver.yaml (e.g., add a new flag)
2. inotify watch on the directory triggers kubelet's file watcher
3. Kubelet detects the change within a few seconds
4. Kubelet reads the new manifest
5. Kubelet stops the existing kube-apiserver container
6. Kubelet starts a new kube-apiserver container with the new configuration
7. Downtime: typically 5-30 seconds depending on container startup time
8. The API server is temporarily unavailable during this window

□ Editing control plane static pod manifests causes temporary API server downtime. In production, do this during maintenance windows and have monitoring in place. Always validate YAML syntax before saving — a YAML syntax error in the manifest prevents the component from starting.

```
# Validate YAML before editing (to avoid syntax errors)
python3 -c 'import yaml; yaml.safe_load(open("/etc/kubernetes/manifests/kube-apiserver.yaml"))'

# Or use kubeval
kubeval /etc/kubernetes/manifests/kube-apiserver.yaml

# Make a backup before editing
cp /etc/kubernetes/manifests/kube-apiserver.yaml /tmp/kube-apiserver.yaml.bak
```

Phase 4 — Logging, Investigation, and the SRE Mindset

Troubleshooting Kubernetes is a skill that develops over time. The difference between a junior engineer and a senior SRE is not the commands they know — it is the investigative methodology they apply. This phase teaches that methodology.

4.1 Where Logs Come From in Kubernetes

Log Source	How to Access It
Pod/Container logs	<code>kubectl logs <pod> [-c <container>] [--previous]</code> . Stored in <code>/var/log/pods/</code> on the node. Also accessible via <code>crictl logs</code> .
kubelet logs	<code>journalctl -u kubelet -f</code> (on the node). Critical for node-level issues: volume mount failures, container startup failures, API server connectivity.
containerd logs	<code>journalctl -u containerd -f</code> . Container runtime issues: image pulls, container starts.
API server logs	<code>kubectl logs -n kube-system kube-apiserver-<node></code> . Or on node: <code>/var/log/pods/kube-system_kube-apiserver-*/kube-apiserver/0.log</code>
etcd logs	<code>kubectl logs -n kube-system etcd-<node></code> . Or <code>journalctl -u etcd</code> (if <code>systemd-managed</code>).
Scheduler logs	<code>kubectl logs -n kube-system kube-scheduler-<node></code>
Controller manager logs	<code>kubectl logs -n kube-system kube-controller-manager-<node></code>
Kubernetes audit logs	Usually at <code>/var/log/kubernetes/audit.log</code> . Shows who did what to the API server.
Node-level syslog	<code>/var/log/syslog</code> or <code>/var/log/messages</code> . Kernel messages, OOM killer events.
dmesg	<code>dmesg -T tail -50</code> . Kernel ring buffer. OOM kills, disk errors, hardware issues.

4.2 The SRE Investigation Framework

Senior SREs follow a disciplined framework. They do not guess — they eliminate possibilities systematically.

9. Start broad, narrow down. First determine: which layer is failing? Application? Pod? Node? Network? Control plane? Storage?
10. Check recent changes. 'What changed?' is almost always the most important question. Check git history, deployment history, node events, cluster events.

11. Gather evidence before acting. Collect logs and state BEFORE making changes. Changes can mask evidence.
12. Form a hypothesis, test it. 'I think CoreDNS is failing because...' Test the specific hypothesis. If wrong, discard it and form a new one.
13. Fix the cause, not the symptom. Restarting a pod is a symptom fix. Finding why it crashed is cause fix.
14. Document as you go. Write down: what you observed, what you tried, what worked. This creates the runbook for next time.

4.3 Pod Not Starting — Complete Debugging Flowchart

```
kubectl get pods → Status: Pending / Error / CrashLoopBackOff / ImagePullBackOff
```

Step 1: `kubectl describe pod <name> -n <namespace>`

Look at:

- Events section (bottom of output) — this is the most important
- Conditions section (Ready, ContainersReady, PodScheduled, Initialized)
- State section (waiting reason, exit code, last state)

PENDING pod — common Events:

- 'Unschedulable: 0/3 nodes available: 3 Insufficient memory'
 - Node resource exhaustion. `kubectl top nodes`
- 'Unschedulable: 0/3 nodes: node affinity mismatch'
 - Check nodeSelector / affinity in pod spec
- 'Unschedulable: had volume node affinity conflict'
 - EBS volume in wrong AZ. See Storage Masterclass Phase 6.
- 'Unschedulable: 0/3 nodes: taint'
 - Node has taint, pod needs matching toleration

IMAGEPULLBACKOFF — common Events:

- 'Failed to pull image: rpc error: code = Unknown... 401 Unauthorized'
 - Image pull secret missing or invalid
 - `kubectl create secret docker-registry`
- 'Failed to pull image: ... not found'
 - Wrong image name or tag. Check typos.
- 'Failed to pull image: context deadline exceeded'
 - Network connectivity to registry. Check DNS, firewall.

CRASHLOOPBACKOFF — most complex:

- `kubectl logs <pod> --previous` (previous container, before crash)
- `kubectl logs <pod>` (current/latest attempt)
- Look for: application errors, missing config, missing secrets
- Check exit code: `kubectl describe pod` → Last State: Terminated → Exit Code
- Exit Code 1 = application error
- Exit Code 137 = OOM killed (check memory limits)
- Exit Code 143 = SIGTERM (graceful shutdown initiated)
- Exit Code 255 = container runtime error

4.4 Node NotReady — Investigation Playbook

```
kubectl get nodes → STATUS: NotReady

Step 1: Identify the node
kubectl describe node <node-name>
  → Check Conditions section
  → MemoryPressure, DiskPressure, PIDPressure, Ready
  → Check events at the bottom

Step 2: SSH to the node
ssh ubuntu@<node-ip>

Step 3: Is kubelet running?
systemctl status kubelet

Step 4: Check kubelet logs for errors
journalctl -u kubelet --since '10 minutes ago' | grep -E 'error|Error|ERROR|fail'

Step 5: Is containerd running?
systemctl status containerd

Step 6: Check disk space
df -h
df -i    # Check inodes

Step 7: Check memory
free -h

Step 8: Check for OOM kills
dmesg -T | grep -i 'oom\\|killed'

Step 9: Check network connectivity to API server
curl -k https://<api-server-ip>:6443/healthz

Step 10: Check CNI
ls /etc/cni/net.d/
ls /opt/cni/bin/
journalctl -u kubelet | grep -i cni
```

4.5 CrashLoopBackOff — Deep Investigation

```
# CrashLoopBackOff means: container starts, crashes, Kubernetes restarts it
# The backoff increases: 10s, 20s, 40s, 80s, 160s, 300s (max)

# Step 1: Check current logs (often empty if container dies immediately)
kubectl logs <pod> -n <namespace>

# Step 2: Check PREVIOUS container logs (most important)
```



```

kubect1 logs <pod> -n <namespace> --previous

# Step 3: Get exit code
kubect1 describe pod <pod> -n <namespace> | grep -A5 'Last State'
# Exit Code 137 = SIGKILL → OOM kill → increase memory limit
# Exit Code 1   = application error → read logs
# Exit Code 2   = application misconfiguration

# Step 4: Override entrypoint to debug interactively
spec:
  containers:
  - name: myapp
    image: myapp:v1
    command: ["/bin/sh"]
    args: ["-c", "sleep 3600"] # Override cmd to prevent crash

# Then: kubect1 exec -it <pod> -- /bin/sh
# Manually run the application inside and see the actual error

```

4.6 Checking Recent Cluster Events

```

# All events sorted by time (last 1 hour)
kubect1 get events --all-namespaces \
  --sort-by='.lastTimestamp' | tail -30

# Events in a specific namespace
kubect1 get events -n production --sort-by='.lastTimestamp'

# Only Warning events
kubect1 get events --all-namespaces \
  --field-selector type=Warning --sort-by='.lastTimestamp'

# Events for a specific resource
kubect1 describe pod <pod-name> # Events section at bottom

```

Phase 5 — YAML Editing and Cluster Reaction

Understanding what happens inside Kubernetes when you apply YAML is fundamental to operational confidence. Different types of changes trigger different reactions — knowing this prevents outages.

5.1 kubectl apply — The Declarative Model

`kubectl apply` is the cornerstone of Kubernetes operations. It implements a 3-way merge: comparing the new manifest (what you want) against the last-applied configuration (what you last applied) and the current live state (what is actually running). Only the differences are applied.

```
# The 3-way merge process
kubectl apply -f deployment.yaml

1. kubectl reads the YAML
2. Compares with kubectl.kubernetes.io/last-applied-configuration annotation
3. Sends a PATCH or PUT to the API server with only the changes
4. API server validates the change
5. API server writes to etcd
6. Relevant controller detects the etcd change (via watch)
7. Controller reconciles: creates/updates/deletes pods to match desired state
```

5.2 Which Changes Require What Level of Disruption

Change Type	Impact and What Happens
Edit Deployment image tag	Deployment controller creates new ReplicaSet with new image. Rolling update: new pods start, old pods terminate. Zero downtime if configured correctly. No restart of Kubernetes components.
Edit Deployment replicas	Controller scales ReplicaSet immediately. New pods scheduled. No restart.
Edit ConfigMap referenced by pod	ConfigMap itself updates instantly. But pods do NOT automatically restart — they keep using the old version (if mounted as volume, Kubernetes eventually updates it, but env vars are fixed at pod start). Must manually restart pods: <code>kubectl rollout restart deployment/<name></code>
Edit Secret	Same as ConfigMap — pods do not auto-restart.
Edit Service ports	Service object updates immediately. kube-proxy on all nodes picks up the change and updates iptables rules within seconds. No pod restart.
Edit HorizontalPodAutoscaler	HPA controller picks up the change. Scale happens if metrics thresholds are met.

Change Type	Impact and What Happens
Edit static pod manifest (e.g., kube-apiserver.yaml)	kubelet detects file change. Stops old container. Starts new container. Causes temporary downtime of that component (~10-30 seconds).
Edit kubelet config (/var/lib/kubelet/config.yaml)	Requires: sudo systemctl restart kubelet. Node NotReady briefly during restart. Pods continue running (kubelet restart does not kill pods).
Edit CNI config (/etc/cni/net.d/)	Requires kubelet restart to pick up. New pods will use new CNI config. Existing pods unaffected until restarted.
Edit /etc/kubernetes/pki/ (certificates)	Requires restart of components that use the cert. API server restart if apiserver cert changed. Extremely high-risk operation.

5.3 Rolling Updates — What Really Happens

```
# Trigger a rolling update
kubectl set image deployment/my-app container=my-app:v2 -n production

# Watch it happen
kubectl rollout status deployment/my-app -n production
kubectl get replicaset -n production -w

# What's happening internally:
# 1. Deployment controller detects spec change
# 2. Creates new ReplicaSet (my-app-7d8f9c) with desired replicas=0
# 3. Scales up new RS by 1 (maxSurge=1 default)
# 4. Waits for new pod to be Ready
# 5. Scales down old RS by 1 (maxUnavailable=0 default → no unavailability)
# 6. Repeats until new RS = desired, old RS = 0

# Rollback if something goes wrong
kubectl rollout undo deployment/my-app -n production

# View rollout history
kubectl rollout history deployment/my-app -n production

# Rollback to specific revision
kubectl rollout undo deployment/my-app --to-revision=2 -n production
```

Phase 6 — 'My Teammate Did a Blunder' — Investigation Scenarios

This is the most practical phase. Real production incidents happen. Here is how senior SREs investigate and recover from the most common Kubernetes blunders.

Scenario 1: Someone Deleted a Namespace

Symptoms: Applications in the namespace are gone. `kubectl get pods -n <namespace>` returns 'No resources found'. PVCs may be deleting.

```
# Immediate investigation
kubectl get namespace <name>
# If Terminating: deletion is in progress

# Check what triggered the deletion
kubectl get events --all-namespaces | grep -i <namespace>

# Check audit logs (if enabled)
grep '"namespace": "<namespace>" *"verb": "delete"' /var/log/kubernetes/audit.log

# RECOVERY:
# If namespace is Terminating and you need to stop it:
# (Emergency only — understand what you're doing)
kubectl get namespace <name> -o json | \
  jq '.spec.finalizers = []' | \
  kubectl replace --raw '/api/v1/namespaces/<name>/finalize' -f -

# Restore from GitOps/Helm
# kubectl apply -f namespace-resources/
# helm install <release> <chart> -n <new-namespace>
```

☐ **If reclaimPolicy: Retain** was set on PVs and storage, the underlying data (EBS volumes, etc.) still exists even after namespace deletion. PVs may be in Released state. You can rebind them to new PVCs.

Scenario 2: CoreDNS is Broken — DNS Not Resolving

Symptoms: Applications cannot connect to other services by name. Errors like 'dial tcp: lookup my-service: no such host'. Pod IPs still work.

```
# Step 1: Is CoreDNS running?
kubectl get pods -n kube-system -l k8s-app=kube-dns

# Step 2: CoreDNS logs
```

```

kubect1 logs -n kube-system -l k8s-app=kube-dns --tail=50

# Step 3: Test DNS from a debug pod
kubect1 run debug --image=busybox:1.35 --rm -it -- sh
# Inside: nslookup kubernetes.default

# Step 4: Check CoreDNS ConfigMap for errors
kubect1 get configmap coredns -n kube-system -o yaml
# A common mistake: someone edited the Corefile incorrectly

# Step 5: Restart CoreDNS
kubect1 rollout restart deployment/coredns -n kube-system

# Step 6: If ConfigMap was corrupted, restore from backup or default
# Default minimal Corefile
apiVersion: v1
kind: ConfigMap
metadata:
  name: coredns
  namespace: kube-system
data:
  Corefile: |
    .:53 {
      errors
      health
      ready
      kubernetes cluster.local in-addr.arpa ip6.arpa {
        pods insecure
        fallthrough in-addr.arpa ip6.arpa
      }
      forward . /etc/resolv.conf
      cache 30
      loop
      reload
      loadbalance
    }

```

Scenario 3: Node Memory Exhausted — Pods Being Evicted

Symptoms: `kubect1 get pods` shows many pods in Evicted state. Applications randomly terminating. `kubect1 describe node` shows `MemoryPressure=True`.

```

# Step 1: Check node resource usage
kubect1 top nodes
kubect1 describe node <overloaded-node>
# Look for: Conditions: MemoryPressure = True
# Look for: Allocatable vs Requests

# Step 2: Which pods are using most memory?
kubect1 top pods --all-namespaces --sort-by=memory | head -20

```

```

# Step 3: Check OOM kill events
dmesg -T | grep -i 'oom\|kill'
# Output: kernel: Out of memory: Kill process 12345 (python) score 900

# Step 4: Find evicted pods (clean them up)
kubectl get pods --all-namespaces --field-selector=status.phase=Failed \
  | grep Evict

# Clean up evicted pods
kubectl get pods --all-namespaces -o json | \
  jq '.items[] | select(.status.reason=="Evicted") | .metadata | [.namespace, .name]' \
  | \
  xargs -n2 sh -c 'kubectl delete pod -n $1 $2' _

# Step 5: Short-term relief – cordon the node
kubectl cordon <node-name>
kubectl drain <node-name> --ignore-daemonsets --delete-emptydir-data

# Step 6: Identify root cause
# Which pod has no memory limit set?
kubectl get pods --all-namespaces -o json | \
  jq '.items[] | select(.spec.containers[].resources.limits.memory == null) | \
  .metadata.name'

```

Scenario 4: etcd Certificate Expired

Symptoms: API server is down. `journalctl -u kubelet` shows certificate errors. `kubectl` commands fail with 'transport: authentication handshake failed: x509: certificate has expired'.

```

# Step 1: Check certificate expiry
kubeadm certs check-expiration

# Step 2: Renew all certificates
kubeadm certs renew all

# Step 3: Restart control plane components
# (Static pods will restart automatically after cert files change)
# If not auto-restarted:
kill -s SIGHUP $(pidof kube-apiserver)

# Step 4: Update kubeconfig
cp /etc/kubernetes/admin.conf ~/.kube/config

# Step 5: Verify
kubectl get nodes

```

Scenario 5: Disk Pressure — Node Disk Full

Symptoms: Node shows `DiskPressure=True`. New pods cannot be scheduled. Existing pods being evicted. `kubectl describe node` shows disk conditions.

```
# Step 1: Find what is using disk
df -h
du -sh /var/log/containers/* | sort -rh | head -20
du -sh /var/lib/docker/overlay2/* | sort -rh | head -10 # if using docker
du -sh /var/lib/containerd/* | sort -rh | head -10

# Step 2: Clean up stopped containers
crictl ps -a | grep Exited | awk '{print $1}' | xargs crictl rm

# Step 3: Clean up unused images
crictl images | awk '{print $3}' # list image IDs
crictl rmi <unused-image-id>

# Step 4: Clean up evicted pods
kubectl delete pods --field-selector=status.phase=Failed --all-namespaces

# Step 5: Check log rotation
ls -lh /var/log/pods/
# If logs are huge, check kubelet log rotation config

# Step 6: Add more disk or move logs to different mount

# Check inode exhaustion (different from disk space)
df -i
# If inodes are full but disk space is available: too many small files
```

Scenario 6: Ingress Not Routing Traffic

Symptoms: `curl https://myapp.company.com` returns 502/503. Pod is running and healthy. Service has endpoints.

```
# Step 1: Check the Ingress object
kubectl describe ingress myapp-ingress -n production
# Check: Address field (should have ELB DNS or node IP)
# Check: TLS section
# Check: Rules (correct host and path?)

# Step 2: Check NGINX Ingress Controller
kubectl get pods -n ingress-nginx
kubectl logs -n ingress-nginx -l app.kubernetes.io/name=ingress-nginx --tail=50

# Step 3: Check backend service endpoints
kubectl get endpoints myapp-service -n production
```

```
# Empty endpoints = selector mismatch or pods not Ready

# Step 4: Test from within the cluster
kubect1 run test --image=curlimages/curl --rm -it -- \
    curl -H 'Host: myapp.company.com' http://<ingress-controller-svc-clusterip>

# Step 5: NGINX config dump
kubect1 exec -n ingress-nginx <nginx-pod> -- cat /etc/nginx/nginx.conf | grep -A10
myapp
```


Phase 7 — Resource Management and QoS

Resource management is the most important factor in Kubernetes cluster stability. Misconfigured resources are the #1 cause of production Kubernetes incidents.

7.1 Requests vs Limits — The Critical Distinction

Property	Requests vs Limits
Definition	Request: the minimum resources Kubernetes guarantees for the container. Used by the scheduler to find a node with enough capacity. Limit: the maximum resources the container is allowed to use. Enforced by cgroups.
CPU Behavior	Request: scheduler ensures node has this much CPU unallocated. Container gets at least this much. Limit: if container tries to use more CPU than its limit, it is throttled (slowed down). CPU is compressible — throttling, not killing.
Memory Behavior	Request: scheduler ensures node has this much memory unallocated. Limit: if container uses more memory than its limit, it is OOM killed immediately (exit code 137). Memory is incompressible — killing, not throttling.
Scheduler Impact	Scheduler uses SUM of requests across all pods to determine if a node can accommodate a new pod. A node with 8GB RAM but 8GB allocated in requests will reject new pods, even if actual usage is 4GB.
Best Practice	Set requests close to typical usage. Set limits at maximum safe usage. Never set limits lower than requests (scheduling will fail). For CPU-sensitive apps, consider no CPU limit (avoid throttling) with node resource limits via LimitRange.

7.2 QoS Classes — How Kubernetes Prioritizes Under Pressure

QoS Class	Criteria and Eviction Priority
Guaranteed	HIGHEST priority. Conditions: All containers have requests AND limits set. CPU request == CPU limit. Memory request == memory limit. These pods are the LAST to be evicted. Use for: critical databases, payment services, control plane workloads.
Burstable	MEDIUM priority. Conditions: At least one container has requests or limits set. But not Guaranteed. These pods are evicted after BestEffort but before Guaranteed. Use for: most production workloads — set requests, allow some burst.

QoS Class	Criteria and Eviction Priority
BestEffort	LOWEST priority. Conditions: No container has any requests or limits set. These pods are the FIRST to be evicted when the node is under memory pressure. Use for: batch jobs, development namespaces. Never for production critical workloads.

```
# Check the QoS class of a pod
kubectl get pod <name> -o jsonpath='{.status.qosClass}'
```

7.3 Node Allocatable vs Capacity

Raw node capacity is never fully available for pods. Kubernetes reserves resources for the OS, kubelet, and system daemons.

```
kubectl describe node <node-name> | grep -A10 'Allocatable\|Capacity'
```

Example output:

```
# Capacity:
#   cpu:      4           # Physical cores
#   memory: 16Gi        # Physical RAM
#   pods:    110         # Max pods per node
# Allocatable:
#   cpu:      3800m       # After kube-reserved and system-reserved
#   memory: 14Gi        # After reservations
#   pods:    110
# Reservations are configured in kubelet config:
# kube-reserved: {cpu: 200m, memory: 512Mi} # for kubelet, kube-proxy
# system-reserved: {cpu: 0, memory: 512Mi} # for OS processes
# evictionThreshold: {memory.available: 100Mi} # trigger eviction
```

7.4 Eviction Thresholds

Kubelet continuously monitors node resources. When resources drop below thresholds, it starts evicting pods to reclaim resources.

Threshold Type	Behavior
Soft eviction	Memory drops below soft threshold. Kubelet gives pods a grace period (default 90s) to shut down. Less disruptive.
Hard eviction	Memory drops below hard threshold (default 100Mi). Kubelet immediately kills pods without grace period. Pods with BestEffort QoS are killed first, then Burstable, then Guaranteed.
DiskPressure	Kubelet evicts pods if disk space drops below threshold. Stops new images from being pulled.

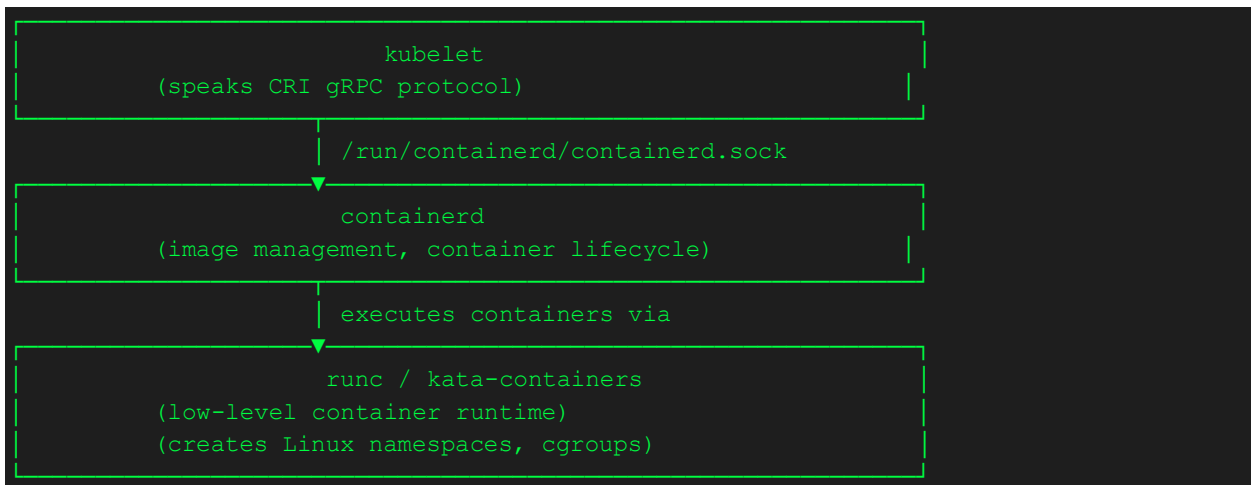
Threshold Type	Behavior
PIDPressure	Too many PIDs on node. Prevents new container processes from spawning.

Phase 8 — Deep Node Troubleshooting

This phase goes to the Linux kernel level — where Kubernetes meets the operating system. Senior SREs must understand what happens below the Kubernetes layer.

8.1 The Container Runtime Layer

Kubernetes does not run containers directly. It talks to a Container Runtime Interface (CRI) implementation — typically containerd or CRI-O. Understanding this layer is essential for deep debugging.



8.2 crictl — The Debugging Tool for Container Runtime

```
# List all running containers
crictl ps

# List all containers including stopped
crictl ps -a

# Get container logs
crictl logs <container-id>

# Inspect a container
crictl inspect <container-id>

# List all pods
crictl pods

# Pull an image
crictl pull nginx:alpine

# List images
```

```
crictl images

# Remove a stuck container
crictl rm <container-id>

# Execute command in container (like docker exec)
crictl exec -it <container-id> /bin/sh
```

8.3 The Pause Container — The Foundation of Pods

Every Kubernetes pod has a special container that you never see in `kubectl get pods` — the pause container (also called the infra container or sandbox container). This is the foundational concept that makes pods work.

When a pod is created:

15. kubelet asks containerd to create a new pod sandbox
16. containerd creates a pause container using a tiny image (`gcr.io/pause:3.x`)
17. This pause container does nothing — it just sleeps forever
18. The pause container creates and holds Linux namespaces: network namespace (one IP per pod), PID namespace, IPC namespace, UTS namespace
19. All application containers in the pod join these existing namespaces
20. All containers in the pod share the same network stack (same IP, same localhost)

```
# See pause containers with crictl
crictl ps | grep pause

# Why does this matter for debugging?
# If your pod has 2 containers sharing localhost:
# Container A serves on :8080
# Container B can reach it via localhost:8080
# This is because they share the network namespace via pause
```

8.4 Linux Namespaces in Kubernetes

Containers are Linux namespaces. Understanding namespaces is essential for advanced debugging.

Namespace Type	What It Isolates and Kubernetes Usage
Network namespace (netns)	Network interfaces, routing tables, firewall rules. Each pod gets its own netns with its own IP. Created by the pause container.
PID namespace	Process ID space. Containers in a pod share PID namespace (by default in Kubernetes, can be enabled per pod). Allows container A to see container B's processes.

Namespace Type	What It Isolates and Kubernetes Usage
Mount namespace	Filesystem mount points. Each container gets its own view of the filesystem. Volume mounts attach external storage to specific paths in this namespace.
UTS namespace	Hostname and domain name. Each container can have its own hostname.
IPC namespace	Inter-process communication (shared memory, semaphores). Pod containers share this for high-performance IPC.
User namespace	User/group ID mapping. Not commonly used in Kubernetes by default.

8.5 nsenter — Entering Container Namespaces

nsenter allows you to enter a container's Linux namespaces from the host. Useful when you need to debug networking inside a pod without exec capability.

```
# Find the container's PID on the host
# Method 1: via crictl
crictl inspect <container-id> | grep pid

# Method 2: via containerd
ctr -n k8s.io containers info <container-id>

# Enter the container's network namespace
nsenter -t <pid> -n ip addr
nsenter -t <pid> -n ip route
nsenter -t <pid> -n ss -tlnp

# Enter all namespaces (like exec)
nsenter -t <pid> -m -u -i -n -p -- /bin/sh

# Run tcpdump inside a container's network namespace
nsenter -t <pid> -n -- tcpdump -i eth0 -w /tmp/capture.pcap
```

8.6 Overlay Networking — How Pod Networks Work

Each pod gets a unique IP from the pod CIDR (e.g., 10.244.0.0/16). This IP is routable within the cluster but not from outside. The CNI plugin creates this network.

```
# View routing on a node
ip route show
# Output:
# 10.244.0.0/24 dev cni0 proto kernel scope link
# 10.244.1.0/24 via 10.0.0.11 dev eth0 ← pods on node-2
# 10.244.2.0/24 via 10.0.0.12 dev eth0 ← pods on node-3
```

```
# View network interfaces
ip link show
# cni0: the bridge connecting pod veth pairs
# veth<id>: virtual ethernet pair connecting pod to bridge
# flannel.1: VXLAN interface (if using Flannel)

# Trace packet path from pod to pod
# Pod A (10.244.0.5) on Node 1 → Pod B (10.244.1.10) on Node 2
# 1. Packet exits pod via veth pair
# 2. Arrives at cni0 bridge on Node 1
# 3. No direct route to 10.244.1.0/24 → uses route via 10.0.0.11
# 4. Packet encapsulated in VXLAN or sent via BGP
# 5. Arrives at Node 2's eth0
# 6. Decapsulated, routed via cni0 to veth pair
# 7. Delivered to Pod B
```

Phase 9 — Production Operations

This phase covers the regular operational activities that keep a production Kubernetes cluster healthy. These are the procedures that experienced SREs perform regularly.

9.1 Daily Cluster Health Checks

```
#!/bin/bash
# Daily cluster health check script

echo '=== Node Status ==='
kubectl get nodes -o wide

echo '=== Node Resource Usage ==='
kubectl top nodes

echo '=== Failed/Error Pods ==='
kubectl get pods --all-namespaces --field-selector=status.phase!=Running,status.phase!=Succeeded

echo '=== Recent Warning Events ==='
kubectl get events --all-namespaces --field-selector=type=Warning \
  --sort-by='.lastTimestamp' | tail -20

echo '=== PVC Status ==='
kubectl get pvc --all-namespaces | grep -v Bound

echo '=== Certificate Expiry ==='
kubeadm certs check-expiration
```

9.2 Node Maintenance — Drain and Cordon

```
# Cordon: mark node as unschedulable (new pods won't be placed here)
# Does NOT evict existing pods
kubectl cordon <node-name>

# Drain: evict all pods from node for maintenance
# Includes: cordon + evict all pods
kubectl drain <node-name> \
  --ignore-daemonsets \           # DaemonSet pods cannot be evicted
  --delete-emptydir-data \       # Delete pods using emptyDir
  --grace-period=60 \            # Give pods 60s to terminate
  --timeout=5m                   # Timeout for drain operation

# After maintenance, uncordon the node
kubectl uncordon <node-name>
```



```
# Safety check: check PodDisruptionBudgets
kubectl get pdb --all-namespaces
# drain will respect PDBs - it will wait if evicting would violate them
```

9.3 etcd Backup — Critical Production Operation

```
# Take etcd snapshot
ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-snapshot-$(date +%Y%m%d-%H%M%S).db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/healthcheck-client.crt \
  --key=/etc/kubernetes/pki/etcd/healthcheck-client.key

# Verify snapshot
ETCDCTL_API=3 etcdctl snapshot status /backup/etcd-snapshot-*.db \
  --write-out=table

# Automate with cron
0 */6 * * * /usr/local/bin/etcd-backup.sh
```

9.4 Certificate Renewal

```
# Check expiry of all certificates
kubeadm certs check-expiration

# Renew all certificates
kubeadm certs renew all

# Certificates renewed to 1 year from now
# Control plane components must be restarted to pick up new certs

# Restart control plane (static pods restart automatically when certs change)
# If not, kill the processes to force kubelet to restart them
kill $(pidof kube-apiserver)
kill $(pidof kube-controller-manager)
kill $(pidof kube-scheduler)

# Update user kubeconfig
cp /etc/kubernetes/admin.conf ~/.kube/config
```

9.5 Production Operations Checklist

- ✓ Daily: `kubectl get nodes`, `kubectl top nodes`, `kubectl get events --field-selector=type=Warning`
- ✓ Daily: Check application health via monitoring dashboards
- ✓ Weekly: Check certificate expiry: `kubeadm certs check-expiration`

- ✓ Weekly: Review PodDisruptionBudgets for all critical workloads
- ✓ Weekly: Verify etcd backups completed successfully
- ✓ Monthly: Rotate service account tokens
- ✓ Monthly: Review RBAC — remove stale bindings
- ✓ Monthly: Node OS patching — drain, patch, uncordon
- ✓ Quarterly: Kubernetes version upgrade plan
- ✓ Quarterly: Disaster recovery drill — restore from etcd backup
- ✓ Quarterly: Chaos engineering — simulate node failure, AZ failure

9.6 The 5-Minute Triage Protocol

When an alert fires at 3 AM, this is the protocol:

21. Establish scope: Is this one pod? One namespace? The whole cluster? `kubectl get nodes && kubectl get pods --all-namespaces | grep -v Running`
22. Check recent events: `kubectl get events --all-namespaces --sort-by='.lastTimestamp' | tail -20`
23. Check control plane health: `kubectl get nodes; curl -k https://localhost:6443/healthz`
24. Identify affected workloads: `kubectl get pods --all-namespaces | grep -v Running`
25. Describe the failing resource: `kubectl describe <resource> <name>` — ALWAYS look at Events section
26. Check logs: `kubectl logs <pod> --previous`
27. Escalate if needed: if control plane is down, SSH to master node and check `journalctl -u kubelet`

Kubernetes Operations & SRE — From Orientation to Expert

Understand the system. Know where to look. Fix fast. Prevent recurrence.